Name: Gaurika

Sap: 590034892

Batch: 15

Experiment-42

Algorithm

Main Algorithm

1. Start.
2. Display the menu:
    - Remove all vowels from a string.
    - Find the first repeating lowercase alphabet.
    - Check whether two strings are anagrams.
    - Exit.
3. Read the user's choice.
4. Use a `switch` statement to perform the selected operation.
5. If choice is **1**, call `removeVowels()`.
6. If choice is **2**, call `firstRepeating()`.
7. If choice is **3**, call `checkAnagram()`.
8. If choice is **4**, exit the program.
9. For any other choice, display `"Invalid choice"`.
10. Repeat the menu until the user selects Exit.
11. Stop.

Algorithm for `removeVowels()`

1. Start the function.
2. Read a string from the user.
3. Initialize `i = 0` and `j = 0`.
4. Traverse the string character by character.
5. Check whether the current character is a vowel (`a, e, i, o, u` in uppercase or lowercase).
6. If it is **not a vowel**, copy it into the result string and increment `j`.
7. Continue until the null character `'\0'` is reached.
8. Add `'\0'` at the end of the result string.
9. Display the string after removing vowels.
10. Return to the main function.

Algorithm for `firstRepeating()`

1. Start the function.
2. Read a string from the user.

3. Declare an integer array `count[26]` and initialize all elements to `0`.
4. Traverse the string from left to right.
5. Check whether the current character is a lowercase alphabet (`a` to `z`).
6. Find its index using `str[i] - 'a'`.
7. Increment the corresponding element of the `count` array.
8. If its count becomes `2`, display that character as the first repeating lowercase alphabet and return.
9. If the entire string is traversed without finding a repeated lowercase alphabet, display `"No repeating lowercase alphabet found"`.
10. Return to the main function.

Algorithm for `checkAnagram()`

1. Start the function.
2. Read two strings from the user.
3. Declare `count[26]` and initialize all elements to `0`.
4. Traverse the first string.
5. For every alphabetic character, convert it to lowercase and increment its corresponding frequency in `count`.
6. Traverse the second string.
7. For every alphabetic character, convert it to lowercase and decrement its corresponding frequency in `count`.
8. Traverse the `count` array.
9. If any element is not `0`, display `"The strings are NOT anagrams"`.
10. If all elements are `0`, display `"The strings are anagrams"`.
11. Return to the main function.

Test Case Table

Test Case	Choice	Input	Expected Output
1	1	`education`	`dctn`
2	1	`Hello World`	`Hll Wrld`
3	1	`AEIOU`	Empty string
4	2	`abcad`	First repeating lowercase alphabet: `a`
5	2	`abcbad`	First repeating lowercase alphabet: `b`
6	2	`abcdef`	No repeating lowercase alphabet found
7	3	`listen`, `silent`	The strings are anagrams
8	3	`Triangle`, `Integral`	The strings are anagrams
9	3	`hello`, `world`	The strings are NOT anagrams
10	4	—	`Exiting program...`
11	5	—	`Invalid choice! Please enter 1-4.`

C Program

```
#include <stdio.h>
```

```
#include <string.h>
#include <ctype.h>

void removeVowels()
{
   char str[100], result[100];
   int i, j = 0;

   printf("Enter a string: ");
   scanf(" %[^\n]", str);

   for (i = 0; str[i] != '\0'; i++)
   {
      if (str[i] != 'a' && str[i] != 'e' &&
         str[i] != 'i' && str[i] != 'o' &&
         str[i] != 'u' && str[i] != 'A' &&
         str[i] != 'E' && str[i] != 'I' &&
         str[i] != 'O' && str[i] != 'U')
      {
         result[j] = str[i];
         j++;
      }
   }

   result[j] = '\0';

   printf("String after removing vowels: %s\n", result);
}

void firstRepeating()
{
```

```
    char str[100];
    int count[26] = {0};
    int i;

    printf("Enter a string: ");
    scanf(" %[^\n]", str);

    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] >= 'a' && str[i] <= 'z')
        {
            count[str[i] - 'a']++;

            if (count[str[i] - 'a'] == 2)
            {
                printf("First repeating lowercase alphabet: %c\n", str[i]);
                return;
            }
        }
    }

    printf("No repeating lowercase alphabet found.\n");
}

void checkAnagram()
{
    char str1[100], str2[100];
    int count[26] = {0};
    int i;

    printf("Enter first string: ");
```

```
    scanf(" %[^\n]", str1);

    printf("Enter second string: ");
    scanf(" %[^\n]", str2);

    for (i = 0; str1[i] != '\0'; i++)
    {
        if (isalpha(str1[i]))
            count[tolower(str1[i]) - 'a']++;
    }

    for (i = 0; str2[i] != '\0'; i++)
    {
        if (isalpha(str2[i]))
            count[tolower(str2[i]) - 'a']--;
    }

    for (i = 0; i < 26; i++)
    {
        if (count[i] != 0)
        {
            printf("The strings are NOT anagrams.\n");
            return;
        }
    }

    printf("The strings are anagrams.\n");
}

int main()
{
```

```
int choice;

do
{
   printf("\n----- MENU -----\n");
   printf("1. Remove all vowels from a string\n");
   printf("2. Find first repeating lowercase alphabet\n");
   printf("3. Check if two strings are anagrams\n");
   printf("4. Exit\n");

   printf("Enter your choice: ");
   scanf("%d", &choice);

   switch (choice)
   {
      case 1:
         removeVowels();
         break;

      case 2:
         firstRepeating();
         break;

      case 3:
         checkAnagram();
         break;

      case 4:
         printf("Exiting program...\n");
         break;
```

```
        default:
            printf("Invalid choice! Please enter 1-4.\n");
    }

} while (choice != 4);

return 0;
}
```

```
┌──(kali㉿kali)-[~]
└─$ vi hdoc42.c

┌──(kali㉿kali)-[~]
└─$ gcc hdoc42.c -o hdoc42

┌──(kali㉿kali)-[~]
└─$ ./hdoc42

──── MENU ────
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
Enter your choice: 1
Enter a string: education
String after removing vowels: dctn

──── MENU ────
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
Enter your choice: 2
Enter a string: abcad
First repeating lowercase alphabet: a
```

```
──── MENU ────
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
Enter your choice: 3
Enter first string: listen,silent
Enter second string: silent
The strings are NOT anagrams.

──── MENU ────
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
Enter your choice: 5
Invalid choice! Please enter 1-4.

──── MENU ────
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
Enter your choice: 4
Exiting program...
```